

SteerNet: An Autonomous Driving agent for Highway-Env

Chen Hao Zhang

Lang Sun

Electrical and Computer Engineering
University of Toronto

December 22, 2025

Abstract

This project tries to solve tasks within the realm of autonomous driving using reinforcement learning (RL) techniques. We choose Highway-Env simulator to be our environment. We plan to use an end to end approach, using image frame extracted from the environment as input to our Deep-Q-Networks. Multiple frames will be stacked together to form the current state of the environment. Evaluation will be carried out using metrics such as cumulative reward, average speed, and crash rate.

Assentation of Teamwork

Chen Hao Zhang: Environment setup and first implementation of DQN for highway-env.

Lang Sun: Investigate and experiment to reduce crash rate.

1 Introduction

Autonomous driving is an exciting field, but writing rule based algorithms to handle complex driving conditions might not be feasible. Reinforcement learning algorithms learn from experience, rather than following explicit instructions.

Highway-env provides multiple forms of state observation, we plan to work with two of them. The first is Kinematics, which returns a matrix that describes the position, velocity of the ego-vehicle, and surrounding vehicles. The second provides a grayscale image.

To answer the question of whether Reinforcement Learning is better than rule-based approaches, we plan to implement a rule-based baseline that drives on simple deterministic rules, then evaluate our trained models against it.

2 Preliminaries and Problem Formulation

The autonomous driving task is formulated as an MDP with the following specifications[3]:

State Representation with Kinematics

The first implementation uses Kinematics where each state consists of a $n \times 5$ observation matrix. Each row represents a vehicle (ego vehicle plus $n - 1$ nearest neighbors) with features:

$$s_t = \begin{bmatrix} p_0 & x_0 & y_0 & v_{x,0} & v_{y,0} \\ p_1 & x_1 & y_1 & v_{x,1} & v_{y,1} \\ p_2 & x_2 & y_2 & v_{x,2} & v_{y,2} \\ p_3 & x_3 & y_3 & v_{x,3} & v_{y,3} \\ p_4 & x_4 & y_4 & v_{x,4} & v_{y,4} \end{bmatrix} \quad (1)$$

where:

- $p_i \in \{0, 1\}$: presence flag for vehicle i (0 for padding)
- (x_i, y_i) : relative position to ego vehicle
- $(v_{x,i}, v_{y,i})$: relative velocity to ego vehicle

In the above example, we set $n = 5$. This fixed-size representation handles variable traffic density through zero-padding when fewer than 5 vehicles are present, or selection of the 4 nearest vehicles when more exist.

State Representation with Grayscale Image

Each frame has a resolution of 240×64 pixels. To incorporate temporal information into the input, we will use frame stacking. A stack of four frames will consist of four consecutive frames.

$$State_dimention = 240 \times 64 \times 4 \quad (2)$$

Action Space

The discrete action set is provided by the Highway-Env simulator:

$$\mathcal{A} = \{\text{LANE_LEFT}, \text{IDLE}, \text{LANE_RIGHT}, \text{FASTER}, \text{SLOWER}\}.$$

Objective

The goal is to train reinforcement learning agents to learn the optimal driving policy $\pi^*(s)$, which maximize expected cumulative reward for all states. In this case, we want to achieve the highest speed, while having the low collision rate.

3 Design

3.1 Baseline Implementation:

In order to evaluate our models, we need to implement a baseline to compare our results. The baseline agent uses simple deterministic rules shown below:

- If the current lane is empty, speed down the current lane
- If the current lane is not empty, check for adjacent lanes
- If there are no empty lanes, follow the car in the front

3.2 DQN using Kinematic Observation

In the kinematic implementation, we use the kinematic matrix, which is $n \times 5$. We set $n = 6$, and flatten it before passing it to the fully connected layers.

The model is a three-layer feedforward neural network. The architecture consists of three fully connected layers:

Linear(feature_size $\rightarrow$ 512), Linear(512 $\rightarrow$ 64), Linear(64 $\rightarrow$ num_actions).

The feature_size is $6 \times 5 = 30$, and the num_actions = 5

3.3 DQN using Grayscale Observation

We use a CNN for both the online network and target network[2].

The CNN takes in an tensor with size of $4 \times 240 \times 64$.

Because a single frame cannot contain temporal information, velocity and direction cannot be inferred from it. Therefore we utilize frame stacking. Four consecutive frames are stacked together, which yields a four-channel input to the CNN. The choice of stack size and kernel size are based on the DeepMind 2015 paper[1].

CNN architecture:

- **Conv1:** Conv2d (4, 32, (8,4), (4,4))
- **Conv2:** Conv2d (32, 64, (4,3), (2,2))
- **Conv3:** Conv2d (64, 64, (3,3), (1,1))
- **FC1: Linear** (conv_out, 512), conv_out = 8320
- **FC2: Linear** (512, 256)
- **FC3: Linear** (256, N_actions), N_actions = 5

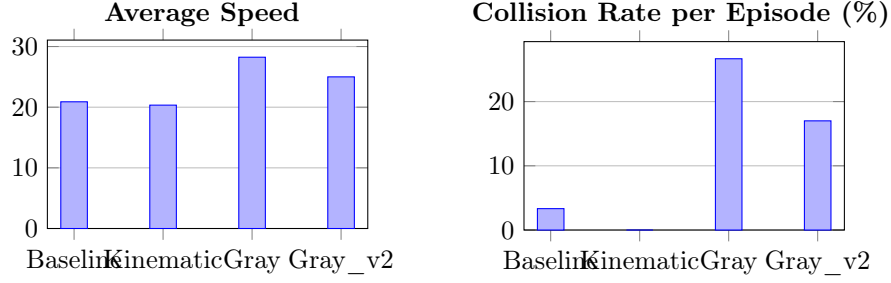


Figure 1: Comparison of average speed and collision rate

3.4 Reward Function

$\text{reward} = \text{crashed} \times \text{crash_penalty}$

$$+ (1 - \text{crashed}) \frac{\text{ego_speed} - \text{min_speed}}{\text{max_speed} - \text{min_speed}}$$

$$+ \begin{cases} \frac{\text{ego_speed}}{19.5}, & \text{if ego_speed} > 19.5, \\ 0, & \text{otherwise,} \end{cases}$$

$$+ \begin{cases} -0.5, & \text{if lane_change,} \\ 0, & \text{otherwise,} \end{cases}$$

$$+ \begin{cases} - \left(1 - \left(\frac{\text{distance_to_front}}{\text{min_safe_distance}} \right)^2 \right) \text{proximity_penalty}, & \text{if distance_to_front} < \text{min_safe_distance} \\ 0, & \text{otherwise.} \end{cases}$$

4 Numerical Experiments

Hyperparameters:

- learning_rate=1e-4
- epsilon_start=1.0
- vehicles_count=20
- gamma=0.9
- epsilon_decay=0.998
- duration=100
- vehicles_density=1.25

5 Discussion: Crash Rate and Experiments

The training plot shows both agents are learning with improving rewards and increasing steps per episode. During evaluation, the Kinematic agent performed similarly to the baseline, While the Grayscale agent is much more aggressive

in its driving strategy. Both the Kinematic and Grayscale agents use the same reward function. However, it is not clear at the moment why the two agents end up having such different driving strategies. In trying to reduce collision rate of the original Grayscale model, we experimented with various training techniques, and our result Grayscale_v2 strikes a balance between speed and collision rate.

5.1 Grayscale Agent with Prioritized Experience Replay (PER)

We build upon our original DQN with Prioritized Experience Replay (PER) [6]. Standard experience replay samples transitions uniformly at random from a buffer, however, not all transitions contribute equally to improving the policy.

PER addresses this by assigning priority to each transition based on its temporal-difference (TD) error, effectively transforming the replay buffer into a priority queue. Transitions with high TD error are sampled more frequently, as these represent experiences the network has not yet learned well.

To correct for the bias introduced by non-uniform sampling, importance sampling weights are applied during gradient updates. However, PER did not meaningfully reduce crash rate.

5.2 Grayscale Agent with PER, Curriculum Learning, and Epsilon Reheating ??

Building upon the Prioritized Experience Replay framework, we implemented a curriculum learning system that decomposes the training process into four stages of increasing complexity. The environment complexity is controlled by varying the number of surrounding vehicles: 5, 10, 15, and 20 cars respectively. The agent advances to the next level upon achieving an average crash rate below 20% over 100 consecutive episodes.

The motivation behind this approach is that policies learned in simpler environments can serve as effective initializations for more complex scenarios.

We also introduced an epsilon reheating mechanism. When the crash rate fails to improve for a specified number of episodes or start to regress, epsilon is increased. This allows the agent to re-explore the environment while still leveraging previously learned behaviors.

With the above changes, the crash rate plateaued at approximately 20-30%, failing to reach our target of 15%.

5.3 Grayscale Agent with PER, Double DQN, and NoisyNet (Grayscale_v2 Agent) ??

Building upon the PER framework, we incorporated Double DQN to address the well-known overestimation of Q-values in standard DQN [4]. In conventional DQN, the same network both selects and evaluates actions for computing target Q-values, which tends to overestimate action values. Double DQN decouples

action selection from action evaluation: the online network selects the best action, while the target network evaluates its value. However, while Double DQN provided modest stability improvements, it did not fundamentally resolve the high crash rate we observed.

Inspired by Fortunato et al. [5], we replaced epsilon-greedy exploration with NoisyNet, which introduces learnable noise parameters into the network. These noise parameters are trained through gradient descent like regular weights. During each forward pass, NoisyNet internally samples two non-learnable noise terms—one for the weights and one for the biases—from a Gaussian distribution. These sampled noise terms are multiplied by the learnable noise parameters to produce the final noisy weights and biases. Because the network’s output depends on this injected noise, the gradient adjusts the noise parameters: if exploration improves performance, gradient descent increases the noise parameters; if exploration increases the loss, gradient descent reduces them. This allows the agent to explore more in unfamiliar states.

Our experiments showed that this approach yielded significant improvements in both the convergence rate and crash rate stability.

6 Conclusions

In conclusion, training an autonomous driving agent using reinforcement learning techniques with pure visual input is feasible. However, the training process is not trivial, and the trained model is often not reliable. For future improvements, a dual loop system where a closed loop subsystem controls the overall driving planning and an open loop subsystem controls the detailed maneuvers might be a better solution for this problem.

References

- [1] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533.
- [2] Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 48:1995-2003.
- [3] Leurent, E. (2018). An environment for autonomous driving decision-making. GitHub repository. <https://github.com/eleurent/highway-env>
- [4] Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [5] Fortunato, M., Azar, M. G., Piot, B., Menick, J., Osband, I., Graves, A., ... & Legg, S. (2017). Noisy networks for exploration. *arXiv preprint arXiv:1706.10295*.
- [6] Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). Prioritized experience replay. *arXiv preprint arXiv:1511.05952*.

Appendix

Appendix A

Project Github Repo: <https://github.com/gh0stintheshe11/SteerNet-v2>

Contest for Information Sharing

As part of this course, we may share selected project materials (e.g., reports, presentation slides, and presentation recordings) on the course webpage as learning resources for future students. Additionally, we may use anonymized project information for internal statistical analysis of course outcomes. Please indicate your preferences below. *Your choices will not affect your grade in any way.*

Consent for Sharing Project Materials

Please keep the item you wish and remove the other one.

- All group members consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

Please keep the item you wish and remove the other one.

- All group members consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Optional Comments

Please list any specific conditions or comments here.

Group Identification

- Group number: 25
- Names of group members: Chen Hao Zhang, Lang Sun
- Signature of of group members: Chen Hao Zhang, Lang Sun
- Date: 2025-Dec-17